Design and Verification of a RISC-V-Integrated BRAM-Backed SRAM Compute-in-Memory Approximation

Rahul Nerella, Uttiyo Majumder, and Kevin Velanjickal
Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
V-SPACE FPGA Build Challenge 2026

***Abstract*—This paper documents an advanced FPGA experiment that connects a single-cycle RV32I processor to a digital compute-in-memory (CiM) engine through a custom instruction decoder and a memory-mapped dispatch controller. The recognized instruction, `CIM.RUN`, uses a reserved custom opcode encoding to launch the PIM engine and select an output row. A control/status interface supports weight and activation loading, busy/done polling, result capture, row-selection clamping, and an explicit dropped-request flag because the base core has no pipeline-stall or backpressure interface. Archived unit-test logs report successful decoder sweeps over 131,072 opcode/funct combinations and 32,768 operand combinations, and successful dispatch tests for loading, a golden result of 272, overlap handling, row clamping, mid-run write rejection, and reset recovery. However, the archived integrated SoC test reports eight failures and unknown program-counter/state values. Subsequent source-level fixes to immediate generation, control-flow targets, and memory indexing are present in the repository but have not been rerun in a simulator. Therefore, this report claims unit-level evidence only and does not claim fully functioning processor-to-CiM integration, synthesis, or physical FPGA validation.**

***Index Terms*—RISC-V, compute-in-memory, SRAM-CiM, custom instruction, dispatch controller, SystemVerilog, FPGA, verification.**

I. Introduction

Neural-network inference repeatedly moves weight and activation values between storage and arithmetic units. Compute-in-memory (CiM) approaches aim to reduce this data movement by moving selected operations closer to the memory array. This experiment explores the digital control and processor-integration side of that idea: a RISC-V core identifies a custom instruction, a dispatch FSM starts a row-wise dot-product engine, and software can stage signed INT8 data and poll completion through a memory-mapped register window.

The project is an FPGA-oriented digital prototype. The bundled `pim_mac_bram` engine reuses the Experiment 4 BRAM-backed PIM approximation; it does not model analog bitline summation, an SRAM bitcell array, ADC nonidealities, or measured silicon energy. The main contribution here is the custom-instruction/dispatch interface and its explicit policy for requests arriving while a single-cycle core continues executing.

II. System Architecture

A. Top-Level Integration

The top module, `soc_top_cim`, connects the RV32I core to instruction/data memories, an existing MAC accelerator address region, and the new CiM path. The data address decoder assigns the `0x1xxxxxxx` region to the retained MAC accelerator and the `0x2xxxxxxx` region to the CiM dispatcher. The custom decoder monitors the current instruction word and emits `cim_valid` and the `rs1` row selector only for the supported instruction encoding.

B. Custom Instruction Encoding

The decoder recognizes one R-type instruction in the RISC-V `custom-0` opcode space. Its identifying fields are:

$$\texttt{opcode} = \texttt{0001011}, \quad \texttt{funct3} = \texttt{001}, \quad \texttt{funct7} = \texttt{0000001}. \tag{1}$$

The instruction's `rs1[4:0]` field selects the result row; `rd` and `rs2` are not used by the dispatch logic. One example word from the testbench is `0x0201100B`, which requests row 2. Because the base processor does not write a result register for this custom instruction, software reads the result through the CiM result MMIO register.

III. Dispatch Controller and Register Interface

A. FSM Operation

The dispatcher uses five states: `D_IDLE`, `D_RUN`, `D_WAIT`, `D_CAP`, and `D_HOLD`. In IDLE and HOLD, software can stage weights and activations. D_RUN provides the one-cycle start pulse to the PIM engine. D_WAIT observes the engine's done signal. D_CAP requests the selected result from the engine's synchronous result port; D_HOLD captures the returned data and exposes a stable result to software.

The launch input follows level-per-instruction semantics rather than rising-edge detection. A single-cycle core advances every cycle and cannot be stalled through the current interface; therefore, another custom instruction may appear while the engine is occupied. Such launches are ignored and set a sticky dropped-request flag. The same policy handles two consecutive `CIM.RUN` words: the second request is flagged instead of being silently merged into the first run. This is a defined

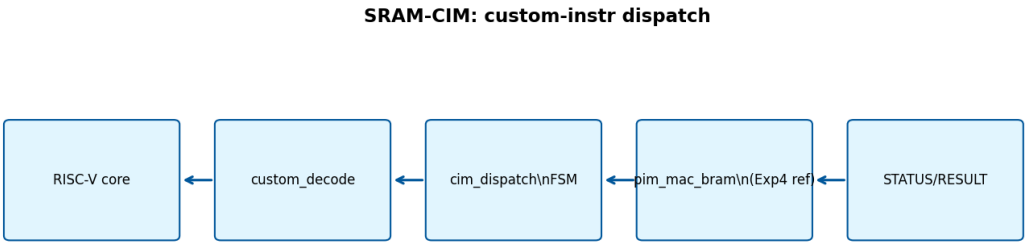


Fig. 1. Project-supplied overview of the RISC-V custom-instruction path. The diagram is conceptual; the implementation details and current verification status are described in the text.

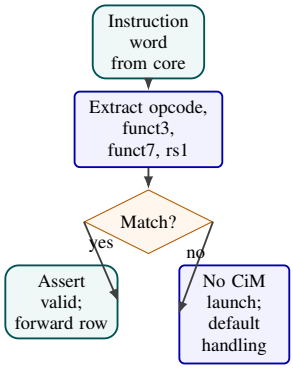


Fig. 2. Custom decoder decision flow. The exhaustive archived decoder test found only the intended opcode/funct combination active.

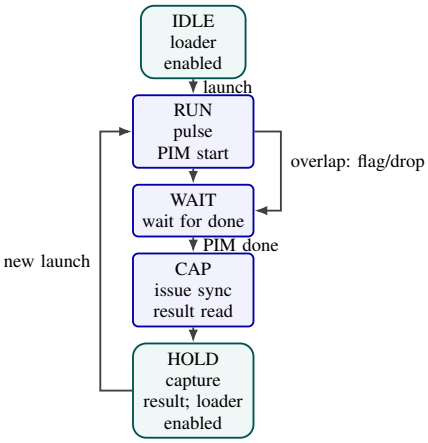


Fig. 3. Dispatch FSM summary. Overlapping requests are flagged and dropped because the base core lacks backpressure.

prototype policy, not a general substitute for pipeline interlocks or an instruction-completion interface.

B. MMIO Map

The default base address is `0x20000000`. Addresses shown in Table I are byte offsets from this base. The implementation drives the result on a combinational bus read while the registered result is captured after the PIM engine's synchronous read latency.

TABLE I
CIM REGISTER WINDOW

Offset	Access	Meaning
`0x00`	R/W	Weight write-address register
`0x04`	W	Weight data; signed INT8 in low byte
`0x08`	R/W	Activation write-address register
`0x0C`	W	Activation data; signed INT8 in low byte
`0x10`	R	Status: bit 0 busy, bit 1 done, bit 2 dropped
`0x14`	R	Selected row's 32-bit result

For a 16-row by 16-column engine, row selection is clamped to the final valid row: `rs1=31` selects row 15. Writes to the weight/activation data ports are accepted only in IDLE/HOLD; writes during computation are ignored. The sticky dropped flag is cleared when a new launch is accepted from an accepting state or when reset is asserted.

IV. COMPUTE MODEL AND FIRMWARE USE

The underlying engine computes signed row-wise dot products:

$$y_r = \sum_{c=0}^{C-1} w_{r,c} a_c, \quad w_{r,c}, a_c \in [-128, 127]. \tag{2}$$

The dispatch prototype defaults to $R = C = 16$ and a 32-bit result path. The testbench loads row 2 with the values $1, 2, \ldots, 16$ and sets each activation to 2, so the expected row-2 result is

$$y_2 = 2 \sum_{c=1}^{16} c = 2 \cdot 136 = 272. \tag{3}$$

Row 15 is loaded with sixteen ones, making its output 32 under the same activation vector. These cases validate basic dispatch, row selection, and result handoff. Firmware is expected to load every weight/activation entry it will use, issue `CIM.RUN`, poll status, and only then read the result.

V. VERIFICATION METHODOLOGY AND RESULTS

The archived run summary records results for three benches. The decoder test sweeps all 128 opcodes, 8 `funct3` values and 128 `funct7` values, for 131,072 combinations, then

TABLE II
ARCHIVED REGRESSION SUMMARY (10 OCTOBER 2026)

Bench	Coverage / observed outcome	Status
Decoder	131,072 opcode/funct combinations; 32,768 operand combinations; near misses	PASS
Dispatch	T1–T6: load/readback, row-2 result 272, dropped flag, clamp result 32, ignored mid-run loads, reset/recovery	PASS
Integrated SoC	CPU firmware, MMIO regression, CiM result, hazard handling, sentinel register, row selector	FAIL: 8 checks

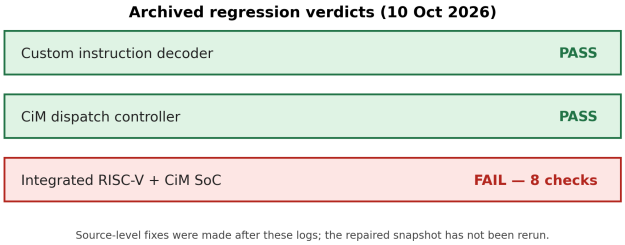


Fig. 4. Status matrix derived from the detailed archived rerun log: decoder and dispatch unit benches passed, while the integrated SoC bench failed eight checks. These are archived results, not a fresh rerun of the repaired source snapshot.

checks 32,768 combinations of `rd`/`rs1`/`rs2` and a set of near-miss encodings. The dispatcher bench checks loader readback, a complete launch-to-result transaction, overlapping launches, sticky status, row clamping, writes during a run, and reset recovery.

The integrated test log reports unknown PC/state values and failures in the MMIO, CiM result, hazard, sentinel-register and row-selection checks. The run notes suggest uninitialized-memory/X propagation as a possible cause, but that explanation is not proven and remains a debugging hypothesis. This paper does not relabel the failed integrated test as a pass.

A. Source Changes Not Yet Reverified

The current repository includes source-level corrections to the RV32I core: sign-extended I/S immediates, shifted U immediates, correct PC-relative branch/jump targets, and PC+4 writeback for JAL/JALR. Instruction and data memory byte addresses are also converted to word indices consistently. These edits target concrete source-review defects, but the historical test logs predate verification of this repaired snapshot. No fresh simulation is claimed here because a compatible simulator was unavailable in the packaging environment. The complete three-bench regression must be run on the current sources before claiming integrated success.

VI. HARDWARE IMPLEMENTATION STATUS

The intended target is an Artix-7 FPGA development board, but the exact allocated board is not identified in the supplied package. There is no experiment-specific XDC file in this experiment folder. Vivado synthesis, place-and-route, timing closure, resource utilization, bitstream programming and board-level demonstration have not been supplied. The

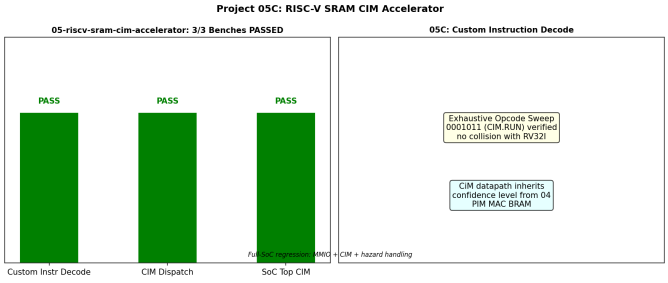


Fig. 5. Legacy result-summary graphic shipped in the project ZIP. It states "3/3 Benches PASSED," which conflicts with the detailed archived rerun log reporting eight integrated-SoC failures. It is shown as a repository artifact, not as the authoritative verification verdict.

BRAM-backed engine is a digital dataflow approximation, not a physical SRAM-CiM macro, and the paper reports no silicon power or energy measurement.

VII. APPLICATIONS AND LIMITATIONS

The design illustrates custom-instruction extension, a memory-mapped accelerator control plane, workload staging, completion polling, and an explicit policy for unsupported overlap hazards. These patterns are useful in teaching heterogeneous SoC integration and in early digital evaluation of accelerator interfaces. Current limitations include a single-cycle core with no stall/backpressure path, dropped-request rather than queued execution, incomplete integrated regression, no synthesized timing/resource data, and no physical SRAM/ADC model.

VIII. CONCLUSION

The supplied archived logs support decoder and dispatch-unit functionality: the decoder sweeps pass, and the dispatcher returns the expected 272 and 32 results while handling busy, overlap, write gating and reset scenarios. They do not support a claim that the complete processor-to-CiM SoC works: the archived integrated test failed eight checks, and subsequent core/memory fixes remain unverified. The next required steps are to rerun all three benches against the exact current RTL, add focused RV32I tests for immediate operations, loads/stores, branches and jumps, resolve the SoC unknown-state failures, add a correct board-specific constraint file, and run synthesis and on-board validation.

REFERENCES

[1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture," including the custom opcode encoding spaces. Available: https://docs.riscv.org/.

[2] "Experiment 5 RTL and testbench," source files `cim_dispatch.sv`, `custom_instr_decode.sv`, `soc_top_cim.sv`, `tb_custom_instr_decode.sv`, `tb_cim_dispatch.sv`, and `tb_soc_top_cim.sv`, project repository.

[3] "Experiment 4: BRAM-backed SRAM PIM approximation," `pim_mac_bram.sv`, V-SPACE FPGA Build Challenge 2026 project repository.

[4] "Archived Experiment 5 regression summary," `Simulation/rerun_2026-10-10.log` and `Simulation/transcript.txt`, 10 October 2026.